

Problems

30-1 Divide-and-conquer multiplication

- a. Show how to multiply two linear polynomials $ax + b$ and $cx + d$ using only three multiplications. (*Hint*: One of the multiplications is $(a + b) \cdot (c + d)$.)
- b. Give two divide-and-conquer algorithms for multiplying two polynomials of degree-bound n in $\Theta(n^{\lg 3})$ time. The first algorithm should divide the input polynomial coefficients into a high half and a low half, and the second algorithm should divide them according to whether their index is odd or even.
- c. Show how to multiply two n -bit integers in $O(n^{\lg 3})$ steps, where each step operates on at most a constant number of 1-bit values.

30-2 Multidimensional fast Fourier transform

The 1-dimensional discrete Fourier transform defined by equation (30.8) generalizes to d dimensions. The input is a d -dimensional array $A = (a_{j_1, j_2, \dots, j_d})$ whose dimensions are $n_1, n_2, \dots, n_d$, where $n_1 n_2 \dots n_d = n$. The d -dimensional discrete Fourier transform is defined by the equation

$$y_{k_1, k_2, \dots, k_d} = \sum_{j_1=0}^{n_1-1} \sum_{j_2=0}^{n_2-1} \dots \sum_{j_d=0}^{n_d-1} a_{j_1, j_2, \dots, j_d} \omega_{n_1}^{j_1 k_1} \omega_{n_2}^{j_2 k_2} \dots \omega_{n_d}^{j_d k_d}$$

for $0 \leq k_1 < n_1, 0 \leq k_2 < n_2, \dots, 0 \leq k_d < n_d$.

- a. Show how to produce a d -dimensional DFT by computing 1-dimensional DFTs on each dimension in turn. That is, first compute n/n_1 separate 1-dimensional DFTs along dimension 1. Then, using the result of the DFTs along dimension 1 as the input, compute n/n_2 separate 1-dimensional DFTs along dimension 2. Using this result as the input, compute n/n_3 separate 1-dimensional DFTs along dimension 3, and so on, through dimension d .

- b.** Show that the ordering of dimensions does not matter, so that if you compute the 1-dimensional DFTs in any order of the d dimensions, you compute the d -dimensional DFT.
- c.** Show that if you compute each 1-dimensional DFT by computing the fast Fourier transform, the total time to compute a d -dimensional DFT is $O(n \lg n)$, independent of d .

30-3 *Evaluating all derivatives of a polynomial at a point*

Given a polynomial $A(x)$ of degree-bound n , we define its t th derivative by

$$A^{(t)}(x) = \begin{cases} A(x) & \text{if } t = 0, \\ \frac{d}{dx} A^{(t-1)}(x) & \text{if } 1 \leq t \leq n-1, \\ 0 & \text{if } t \geq n. \end{cases}$$

In this problem, you will show how to determine $A^{(t)}(x_0)$ for $t = 0, 1, \dots, n-1$, given the coefficient representation $(a_0, a_1, \dots, a_{n-1})$ of $A(x)$ and a point x_0 .

- a.** Given coefficients $b_0, b_1, \dots, b_{n-1}$ such that

$$A(x) = \sum_{j=0}^{n-1} b_j (x - x_0)^j,$$

show how to compute $A^{(t)}(x_0)$, for $t = 0, 1, \dots, n-1$, in $O(n)$ time.

- b.** Explain how to find $b_0, b_1, \dots, b_{n-1}$ in $O(n \lg n)$ time, given $A(x_0 + \omega_n^k)$ for $k = 0, 1, \dots, n-1$.

- c.** Prove that

$$A(x_0 + \omega_n^k) = \sum_{r=0}^{n-1} \left(\frac{\omega_n^{kr}}{r!} \sum_{j=0}^{n-1} f(j) g(r-j) \right),$$

where $f(j) = a_j \cdot j!$ and

$$g(l) = \begin{cases} x_0^{-l} / (-l)! & \text{if } -(n-1) \leq l \leq 0, \\ 0 & \text{if } 1 \leq l \leq n-1. \end{cases}$$

- d.** Explain how to evaluate $A(x_0 + \omega_n^k)$ for $k = 0, 1, \dots, n-1$ in $O(n \lg n)$ time. Conclude that you can evaluate all nontrivial derivatives of $A(x)$ at x_0 in $O(n \lg n)$ time.

30-4 Polynomial evaluation at multiple points

Problem 2-3 showed how to evaluate a polynomial of degree-bound n at a single point in $O(n)$ time using Horner's rule. This chapter described how to evaluate such a polynomial at all n complex roots of unity in $O(n \lg n)$ time using the FFT. Now, you will show how to evaluate a polynomial of degree-bound n at n arbitrary points in $O(n \lg^2 n)$ time.

To do so, assume that you can compute the polynomial remainder when one such polynomial is divided by another in $O(n \lg n)$ time. For example, the remainder of $3x^3 + x^2 - 3x + 1$ when divided by $x^2 + x + 2$ is

$$(3x^3 + x^2 - 3x + 1) \bmod (x^2 + x + 2) = -7x + 5.$$

Given the coefficient representation of a polynomial $A(x) = \sum_{k=0}^{n-1} a_k x^k$ and n points $x_0, x_1, \dots, x_{n-1}$, your goal is to compute the n values $A(x_0), A(x_1), \dots, A(x_{n-1})$. For $0 \leq i \leq j \leq n-1$, define the polynomials $P_{ij}(x) = \prod_{k=i}^j (x - x_k)$ and $Q_{ij}(x) = A(x) \bmod P_{ij}(x)$. Note that $Q_{ij}(x)$ has degree at most $j - i$.

- a.** Prove that $A(x) \bmod (x - z) = A(z)$ for any point z .
- b.** Prove that $Q_{kk}(x) = A(x_k)$ and that $Q_{0,n-1}(x) = A(x)$.
- c.** Prove that for $i \leq k \leq j$, we have both $Q_{ik}(x) = Q_{ij}(x) \bmod P_{ik}(x)$ and $Q_{kj}(x) = Q_{ij}(x) \bmod P_{kj}(x)$.
- d.** Give an $O(n \lg^2 n)$ -time algorithm to evaluate $A(x_0), A(x_1), \dots, A(x_{n-1})$.

30-5 FFT using modular arithmetic

As defined, the discrete Fourier transform requires computation with complex numbers, which can result in a loss of precision due to round-off errors. For some problems, the answer is known to contain only integers, and a variant of the FFT based on modular arithmetic can guarantee that the answer is calculated exactly. An example of such a problem is that of multiplying two polynomials with integer coefficients. Exercise 30.2-6 gives one approach, using a modulus of length $\Omega(n)$ bits to handle a DFT on n points. This problem explores another approach that uses a modulus of the more reasonable length $O(\lg n)$, but it requires that you understand the material of Chapter 31. Let n be an exact power of 2.

- a.** You wish to search for the smallest k such that $p = kn + 1$ is prime. Give a simple heuristic argument why you might expect k to be approximately $\ln n$. (The value of k might be much larger or smaller, but you can reasonably expect to examine $O(\lg n)$ candidate values of k on average.) How does the expected length of p compare to the length of n ?

Let g be a generator of $\mathbb{Z}_p^*$, and let $w = g^k \bmod p$.

- b.** Argue that the DFT and the inverse DFT are well-defined inverse operations modulo p , where w is used as a principal n th root of unity.
- c.** Show how to make the FFT and its inverse work modulo p in $O(n \lg n)$ time, where operations on words of $O(\lg n)$ bits take unit time. Assume that the algorithm is given p and w .
- d.** Compute the DFT modulo $p = 17$ of the vector $(0, 5, 3, 7, 7, 2, 1, 6)$. (*Hint:* Verify and use the fact that $g = 3$ is a generator of $\mathbb{Z}_{17}^*$.)

Chapter notes

Van Loan's book [442] provides an outstanding treatment of the fast Fourier transform. Press, Teukolsky, Vetterling, and Flannery [365, 366] offer a good description of the fast Fourier transform and its applications. For an excellent introduction to signal processing, a popular FFT application area, see the texts by Oppenheim and Schaffer

[347] and Oppenheim and Willsky [348]. The Oppenheim and Schaffer book also shows how to handle cases in which n is not an exact power of 2.

Fourier analysis is not limited to 1-dimensional data. It is widely used in image processing to analyze data in two or more dimensions. The books by Gonzalez and Woods [194] and Pratt [363] discuss multidimensional Fourier transforms and their use in image processing, and books by Tolimieri, An, and Lu [439] and Van Loan [442] discuss the mathematics of multidimensional fast Fourier transforms.

Cooley and Tukey [101] are widely credited with devising the FFT in the 1960s. The FFT had in fact been discovered many times previously, but its importance was not fully realized before the advent of modern digital computers. Although Press, Teukolsky, Vetterling, and Flannery attribute the origins of the method to Runge and König in 1924, an article by Heideman, Johnson, and Burrus [211] traces the history of the FFT as far back as C. F. Gauss in 1805.

Frigo and Johnson [161] developed a fast and flexible implementation of the FFT, called FFTW (“fastest Fourier transform in the West”). FFTW is designed for situations requiring multiple DFT computations on the same problem size. Before actually computing the DFTs, FFTW executes a “planner,” which, by a series of trial runs, determines how best to decompose the FFT computation for the given problem size on the host machine. FFTW adapts to use the hardware cache efficiently, and once subproblems are small enough, FFTW solves them with optimized, straight-line code. Moreover, FFTW has the advantage of taking $\Theta(n \lg n)$ time for any problem size n , even when n is a large prime.

Although the standard Fourier transform assumes that the input represents points that are uniformly spaced in the time domain, other techniques can approximate the FFT on “nonequispaced” data. The article by Ware [449] provides an overview.

¹ Interpolation is a notoriously tricky problem from the point of view of numerical stability. Although the approaches described here are mathematically correct, small differences in the inputs or round-off errors during computation can cause large differences in the result.

² Many other authors define ω_n differently: $\omega_n = e^{-2\pi i/n}$. This alternative definition tends to be used for signal-processing applications. The underlying mathematics is substantially the same with either definition of ω_n .

³ The length n is actually what Section 30.1 referred to as $2n$, since the degree-bound of the given polynomials doubles prior to evaluation. In the context of polynomial multiplication, therefore, we are actually working with complex $(2n)$ th roots of unity.

⁴ The downside of iteratively updating ω is that round-off errors can accumulate, especially for larger input sizes. Several techniques to limit the magnitude of FFT round-off errors have been proposed, but are beyond the scope of this book. If several FFTs are going to be run on inputs of the same size, then it might be worthwhile to directly precompute a table of all $n/2$ values of ω_n^k .